



ADVANCED CYBERSECURITY PRACTICAL ASSESSMENT REPORT

Network Security Assessment & Traffic Investigation

Submitted By	Avisha Masih
Role	Cybersecurity Intern
Organization	DG Interns Hub
Week	Week 3
Submission Date	8th September 2026

Table of Contents

1. Introduction.....	03
2. Objectives.....	03
3. Lab Environment.....	04
4. Network Diagram.....	04
5. Tools Used.....	05
6. Nmap Methodology.....	06-07
7. Nmap Results.....	08
8. Advanced Nmap Analysis.....	09-14
9. Wireshark Methodology.....	15-16
10. Packet Analysis.....	17-18
11. Nmap + Wireshark Investigation.....	19-20
12. Vulnerability Assessment.....	21-23
13. Risk Classification.....	24
14. Security Hardening.....	24-25
15. Final Mini Security Assessment.....	26
16. Before/After Comparison.....	27
17. Security Recommendations.....	28
18. Challenges Faced.....	28
19. Conclusion.....	29
20. References.....	29
21. Screenshots / Evidence.....	29

1. Introduction

This report documents the practical work completed during Week 3 of the Cybersecurity Internship program. The focus of this week was on real-world network security assessment techniques, including **host discovery**, **port scanning**, **service enumeration**, **network traffic analysis**, **vulnerability identification**, and **security hardening**.

All activities were performed in a controlled, authorized virtual lab environment using **VirtualBox**, ensuring that no unauthorized systems or third-party networks were accessed or scanned. The primary tools used were **Nmap** for network scanning and **Wireshark** for packet-level traffic analysis.

This report presents the complete methodology, findings, evidence, and recommendations derived from the Week 3 tasks.

2. Objectives

The objectives of this Week 3 practical assignment were to:

1. Perform **network discovery** to identify live hosts within an authorized lab network.
2. Conduct basic and advanced **Nmap** scans to identify open ports, running services, and their versions.
3. Capture and analyze network traffic using **Wireshark** to understand protocol behavior (**DNS, TCP, UDP, ICMP, ARP, HTTP**).
4. Correlate Nmap scan activity with corresponding Wireshark packet captures to understand scanning at the packet level.
5. Investigate captured traffic for suspicious or noteworthy patterns.
6. Perform a basic **vulnerability assessment** based on findings from scanning and traffic analysis.
7. Apply **security hardening** measures to reduce the identified attack surface and verify improvements through before/after testing.
8. Document the entire process in a professional security assessment format.

3. Lab Environment

The lab environment was built using **Oracle VirtualBox** as the virtualization platform. The following virtual machines were used:

VM Name	OS	Role	IP Address	Network Mode
Kali Linux	Kali Linux 2026.2	Analyst / Attacker Machine (Nmap + Wireshark)	10.0.2.15	NAT (VirtualBox)
Ubuntu-Target	Ubuntu Server 26.04.1 LTS	Target Machine (SSH service)	10.0.2.3	NAT (VirtualBox)

All scanning and traffic capture activities were performed strictly within this isolated, authorized virtual network. No external or public IP addresses were scanned at any point during this assessment.

4. Network Diagram

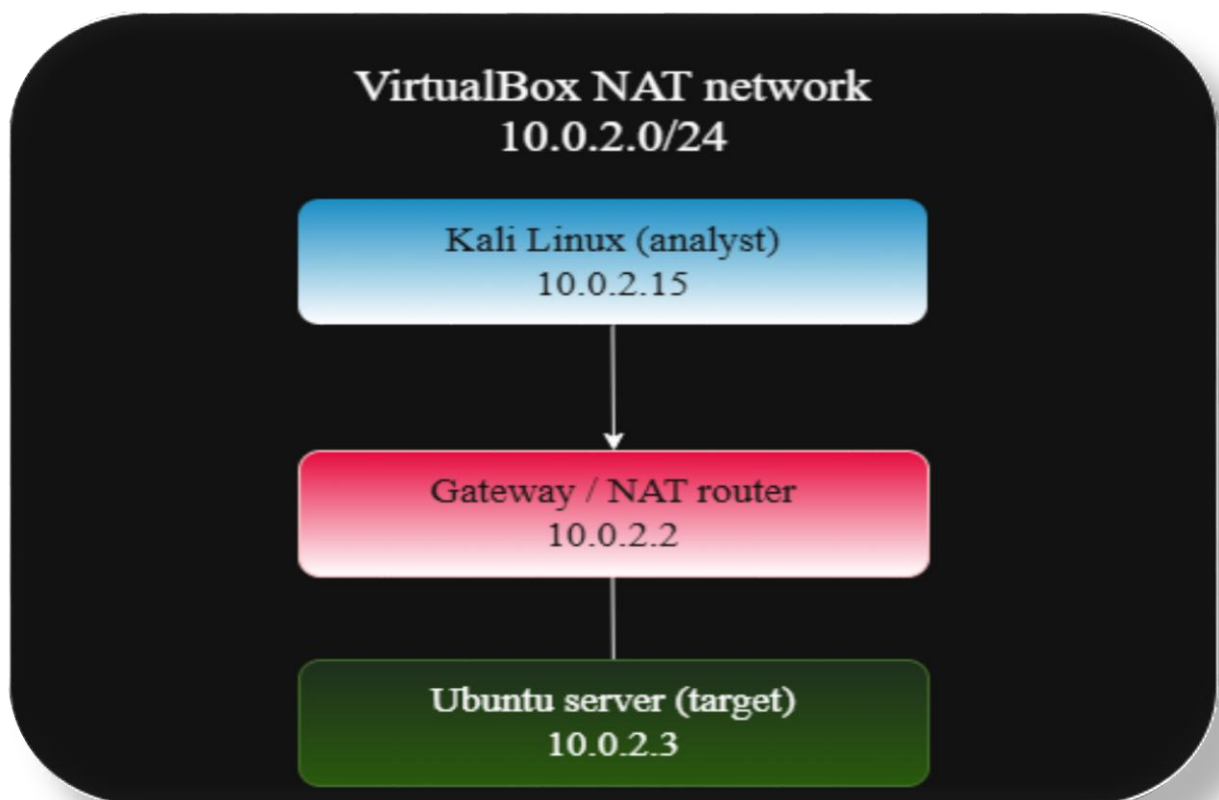


Fig 1: Lab Network Topology Diagram

5. Tools Used

Oracle VirtualBox – Virtualization platform used to create and manage isolated virtual machines for the lab environment.

Kali Linux 2026.2 – Penetration testing operating system used as the primary analyst/attacker machine, pre-loaded with Nmap and Wireshark.

Ubuntu Server 26.04.1 LTS – Target machine used to simulate a real-world server exposing common network services (SSH, HTTP) for scanning and assessment purposes.

Nmap (Network Mapper) – Used for host discovery, port scanning, service/version detection, OS detection, and vulnerability scanning via NSE scripts.

Wireshark – Used for capturing and analyzing network traffic at the packet level, including DNS, TCP, UDP, ICMP, and ARP protocols.

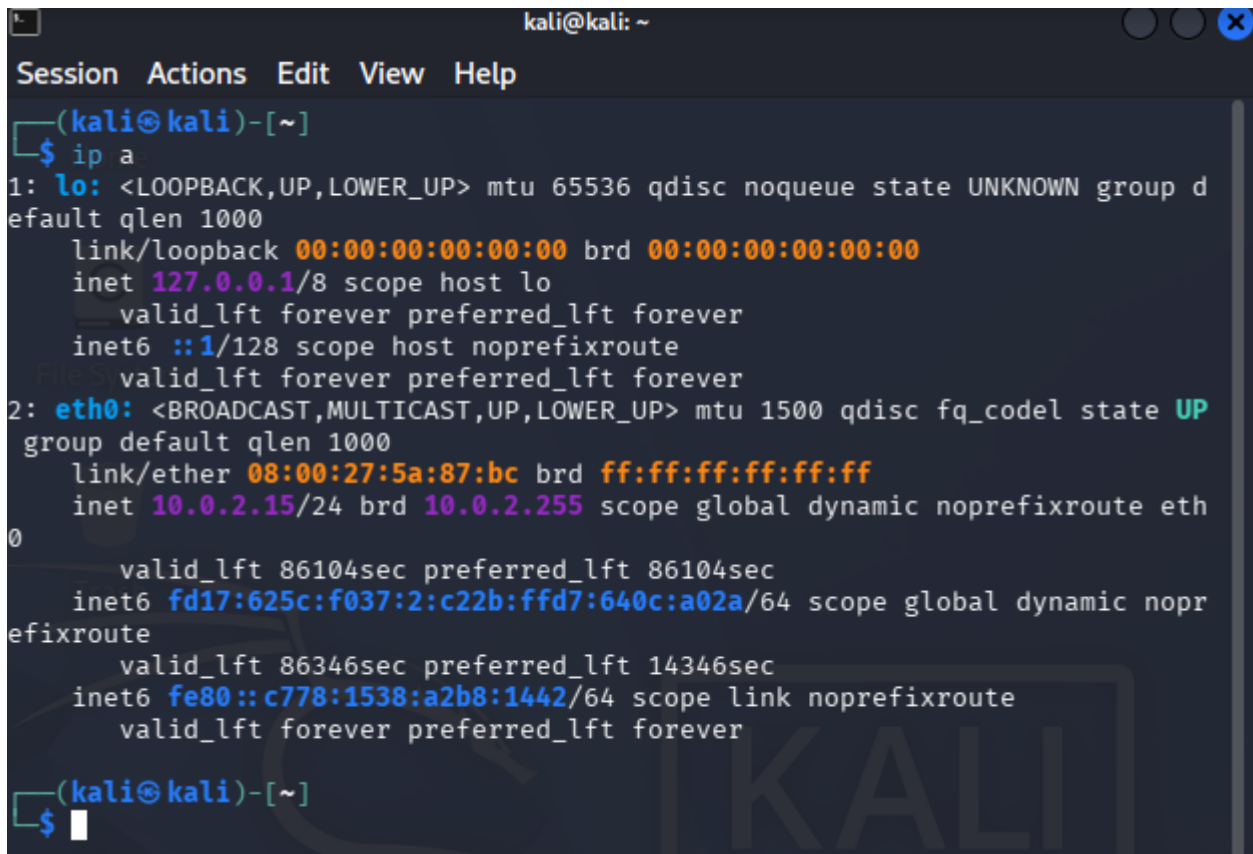
draw.io (diagrams.net) – Used to create the network topology diagram representing the lab environment.

GitHub – Used for version control and hosting the final project documentation and files.

6. Nmap Methodology (Task 7 & 8)

6.1 Network Discovery

The lab network configuration was first identified on the Kali Linux analyst machine using the 'ip a' command. This revealed an IP address of **10.0.2.15** on interface **eth0**, with a /24 subnet mask, operating under VirtualBox's NAT network. The default gateway was then identified using the 'ip route' command, which returned **10.0.2.2** as the **gateway** address.



```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86104sec preferred_lft 86104sec
    inet6 fd17:625c:f037:2:c22b:ffd7:640c:a02a/64 scope global dynamic noprefixroute
        valid_lft 86346sec preferred_lft 14346sec
    inet6 fe80::c778:1538:a2b8:1442/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(kali@kali)-[~]
$
```

Fig 2: ip a output showing Kali IP & interface

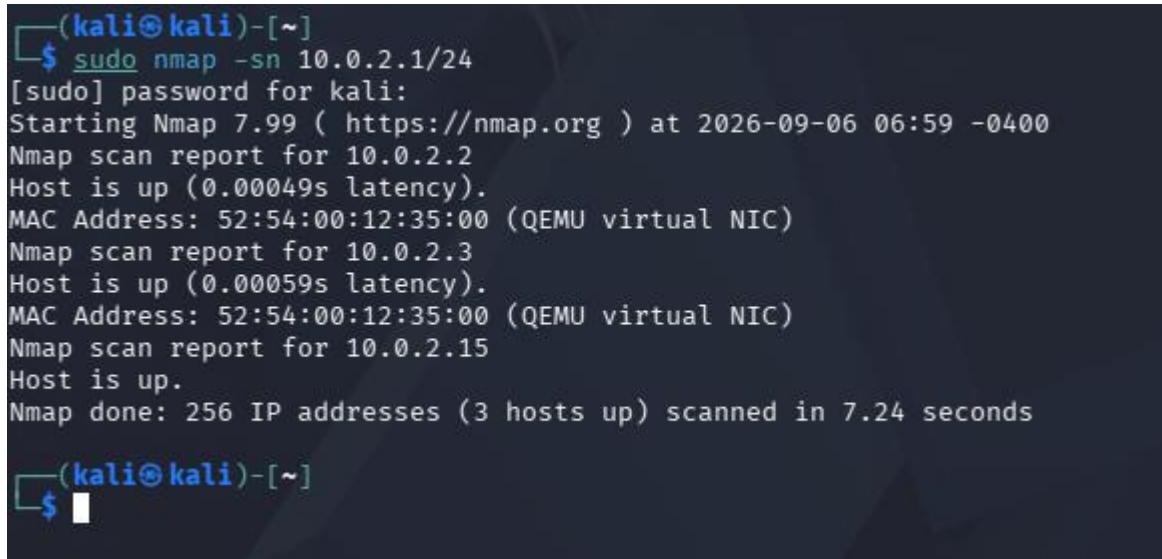


```
(kali@kali)-[~]
$ ip route
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
(kali@kali)-[~]
$
```

Fig 3: ip route output showing default gateway

6.2 Host Discovery

A host discovery (ping) scan was performed across the full lab subnet using the command: **nmap -sn 10.0.2.0/24**. This scan does not probe ports; it only determines which hosts on the network are currently online. The scan completed in 7.24 seconds and identified **3 live hosts** out of 256 possible addresses in the subnet.

A terminal window on a Kali Linux system showing the execution of an nmap host discovery scan. The user runs 'sudo nmap -sn 10.0.2.1/24'. The terminal output shows the scan starting at 2026-09-06 06:59 -0400. It reports three hosts as up: 10.0.2.2, 10.0.2.3, and 10.0.2.15. Each host has a latency of approximately 0.00049s to 0.00059s and a MAC address of 52:54:00:12:35:00 (QEMU virtual NIC). The scan is completed in 7.24 seconds, having scanned 256 IP addresses.

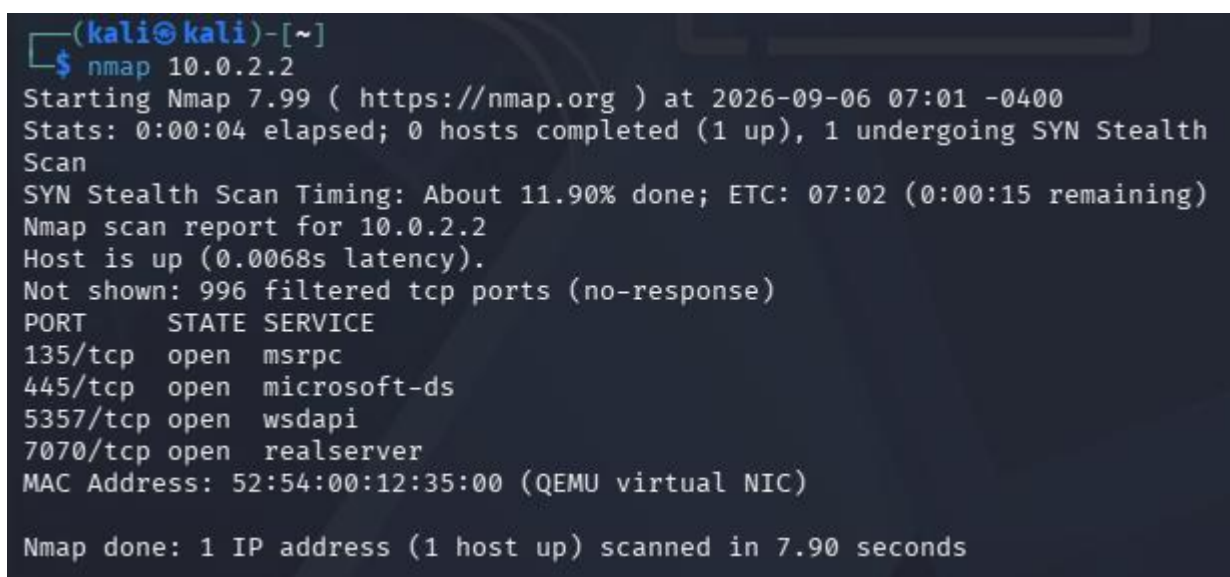
```
(kali@kali)-[~]
$ sudo nmap -sn 10.0.2.1/24
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 06:59 -0400
Nmap scan report for 10.0.2.2
Host is up (0.00049s latency).
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap scan report for 10.0.2.3
Host is up (0.00059s latency).
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap scan report for 10.0.2.15
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 7.24 seconds

(kali@kali)-[~]
$
```

Fig 4: nmap -sn 10.0.2.0/24 host discovery result

6.3 Basic Scan

A basic Nmap scan was then run against the discovered gateway host (10.0.2.2) using the command: **nmap 10.0.2.2**. This performed a default SYN-based scan of the most common 1000 ports, revealing **4 open ports** out of 1000 scanned (**996** were **filtered**, meaning no response was received for those ports, indicating a firewall is present).

A terminal window on a Kali Linux system showing the execution of a basic nmap scan against 10.0.2.2. The user runs 'nmap 10.0.2.2'. The terminal output shows the scan starting at 2026-09-06 07:01 -0400. It reports that the host is up with a latency of 0.0068s. A SYN Stealth Scan is in progress, with 11.90% completion. The scan report for 10.0.2.2 shows four open ports: 135/tcp (msrpc), 445/tcp (microsoft-ds), 5357/tcp (wsdapi), and 7070/tcp (realserver). The remaining 996 ports are filtered. The scan is completed in 7.90 seconds.

```
(kali@kali)-[~]
$ nmap 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 07:01 -0400
Stats: 0:00:04 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 11.90% done; ETC: 07:02 (0:00:15 remaining)
Nmap scan report for 10.0.2.2
Host is up (0.0068s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
5357/tcp   open  wsdapi
7070/tcp   open  realserver
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 7.90 seconds
```

Fig 5: nmap 10.0.2.2 basic scan result

7. Nmap Results (Task 7)

Discovered hosts on the lab network (from **nmap -sn 10.0.2.0/24**):

Device	IP Address	Status
Default Gateway / NAT Router	10.0.2.2	Up
Secondary Host (Ubuntu-Target)	10.0.2.3	Up
Kali Linux (Analyst Machine)	10.0.2.15	Up

Basic scan results on target host 10.0.2.2 (from **nmap 10.0.2.2**):

IP Address	Open Port	Protocol	Service
10.0.2.2	135	TCP	msrpc
10.0.2.2	445	TCP	microsoft-ds
10.0.2.2	5357	TCP	wsdapi
10.0.2.2	7070	TCP	realserver

Of the 1000 most common ports scanned, **996** were reported as filtered (no response received, indicating a firewall), and **4** were found **open**, as listed above.

8. Advanced Nmap Analysis (Task 8)

Building on the basic scan from Task 7, a series of advanced Nmap scans were performed against the target host (10.0.2.2) to gather deeper information about running services, the operating system, and potential vulnerabilities.

8.1 Service and Version Detection (nmap -sV)

Version detection identified the exact software running behind each open port: Microsoft Windows RPC on **port 135**, microsoft-ds on **port 445**, Microsoft HTTPAPI httpd 2.0 (used for SSDP/UPnP) on **port 5357**, and an ssl/realservice-like service on **port 7070**. The scan also fingerprinted the underlying OS as Windows based on service characteristics (CPE: cpe:/o:microsoft:windows).

```
(kali㉿kali)-[~]
$ sudo nmap -sV 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 07:11 -0400
Nmap scan report for 10.0.2.2
Host is up (0.0050s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
445/tcp    open  microsoft-ds?
5357/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
7070/tcp   open  ssl/realservice?
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

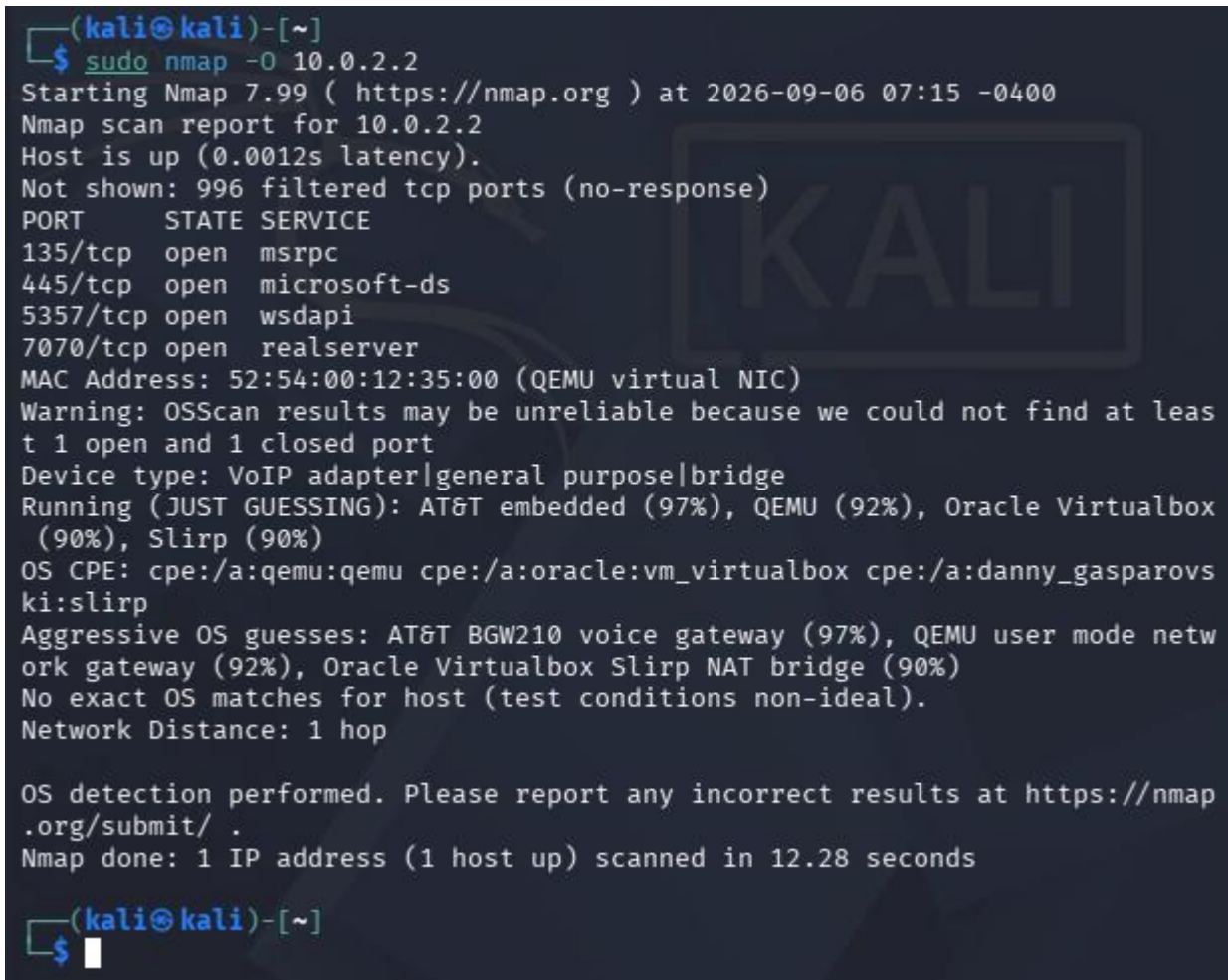
Service detection performed. Please report any incorrect results at https://
/nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.59 seconds

(kali㉿kali)-[~]
$
```

Fig 6: nmap -sV service/version detection output

8.2 OS Detection (nmap -O)

OS fingerprinting returned a best guess of 'Oracle VirtualBox Slirp NAT bridge' (**90% confidence**), alongside AT&T embedded and QEMU general-purpose guesses. Nmap explicitly noted that no exact OS match was found and that results may be unreliable, since only one port was open and closed on the host used for fingerprinting. This is expected behavior when scanning a virtualized NAT gateway rather than a full physical host, and is a useful real-world observation: automated OS detection is inherently probabilistic in virtualized/NAT environments.



```
(kali㉿kali)-[~]
$ sudo nmap -O 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 07:15 -0400
Nmap scan report for 10.0.2.2
Host is up (0.0012s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
5357/tcp   open  wsddapi
7070/tcp   open  realserver
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: VoIP adapter|general purpose|bridge
Running (JUST GUESSING): AT&T embedded (97%), QEMU (92%), Oracle Virtualbox (90%), Slirp (90%)
OS CPE: cpe:/a:qemu:qemu cpe:/a:oracle:vm_virtualbox cpe:/a:danny_gasparovs:ki:slirp
Aggressive OS guesses: AT&T BGW210 voice gateway (97%), QEMU user mode network gateway (92%), Oracle Virtualbox Slirp NAT bridge (90%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

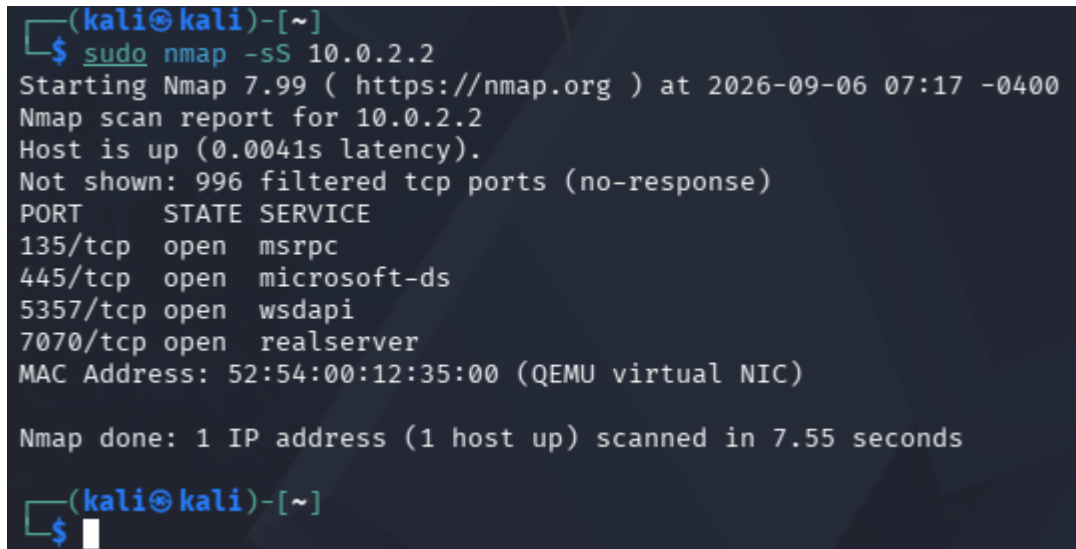
OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.28 seconds

(kali㉿kali)-[~]
$
```

Fig 7: nmap -O OS detection output

8.3 TCP SYN Scan (nmap -sS)

A stealth SYN scan confirmed the same 4 open ports (**135, 445, 5357, 7070**) found in the earlier scans, with 996 ports reported as **filtered** (no response), indicating a firewall or packet-filtering device is present on the path to the target.

A terminal window on a Kali Linux system showing the output of an nmap SYN scan. The prompt is (kali@kali)-[~]. The command executed is \$ sudo nmap -sS 10.0.2.2. The output shows the scan starting at 2026-09-06 07:17 -0400, reporting 4 open ports (135/tcp, 445/tcp, 5357/tcp, 7070/tcp) and 996 filtered ports. The MAC address is 52:54:00:12:35:00 (QEMU virtual NIC). The scan took 7.55 seconds.

```
(kali@kali)-[~]  
$ sudo nmap -sS 10.0.2.2  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 07:17 -0400  
Nmap scan report for 10.0.2.2  
Host is up (0.0041s latency).  
Not shown: 996 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
135/tcp   open  msrpc  
445/tcp   open  microsoft-ds  
5357/tcp  open  wsapi  
7070/tcp  open  realserver  
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 7.55 seconds  
  
(kali@kali)-[~]  
$
```

Fig 8: nmap -sS SYN scan output

8.4 UDP Scan (nmap -sU --top-ports 20)

A UDP scan of the top 20 UDP ports found one clearly open port (**69/udp - tftp**), and three ports reported as open|filtered (**67/udp - dhcp**, **137/udp - netbios-ns**, **4500/udp - nat-t-ike**). An open|filtered state means Nmap received no response and cannot conclusively determine whether the port is open or blocked by a firewall; this is a common and expected result for UDP scanning, since UDP does not have the equivalent of a reliable TCP handshake to confirm state.

```
(kali@kali)-[~]
$ sudo nmap -sU --top-ports 20 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 07:18 -0400
Nmap scan report for 10.0.2.2
Host is up (0.0015s latency).

PORT      STATE      SERVICE
53/udp    closed    domain
67/udp    open|filtered dhcp
68/udp    closed    dhcp
69/udp    open      tftp
123/udp   closed    ntp
135/udp   closed    msrpc
137/udp   open|filtered netbios-ns
138/udp   closed    netbios-dgm
139/udp   closed    netbios-ssn
161/udp   closed    snmp
162/udp   closed    snmptrap
445/udp   closed    microsoft-ds
500/udp   closed    isakmp
514/udp   closed    syslog
520/udp   closed    route
631/udp   closed    ipp
1434/udp  closed    ms-sql-m
1900/udp  closed    upnp
4500/udp  open|filtered nat-t-ike
49152/udp closed    unknown
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)

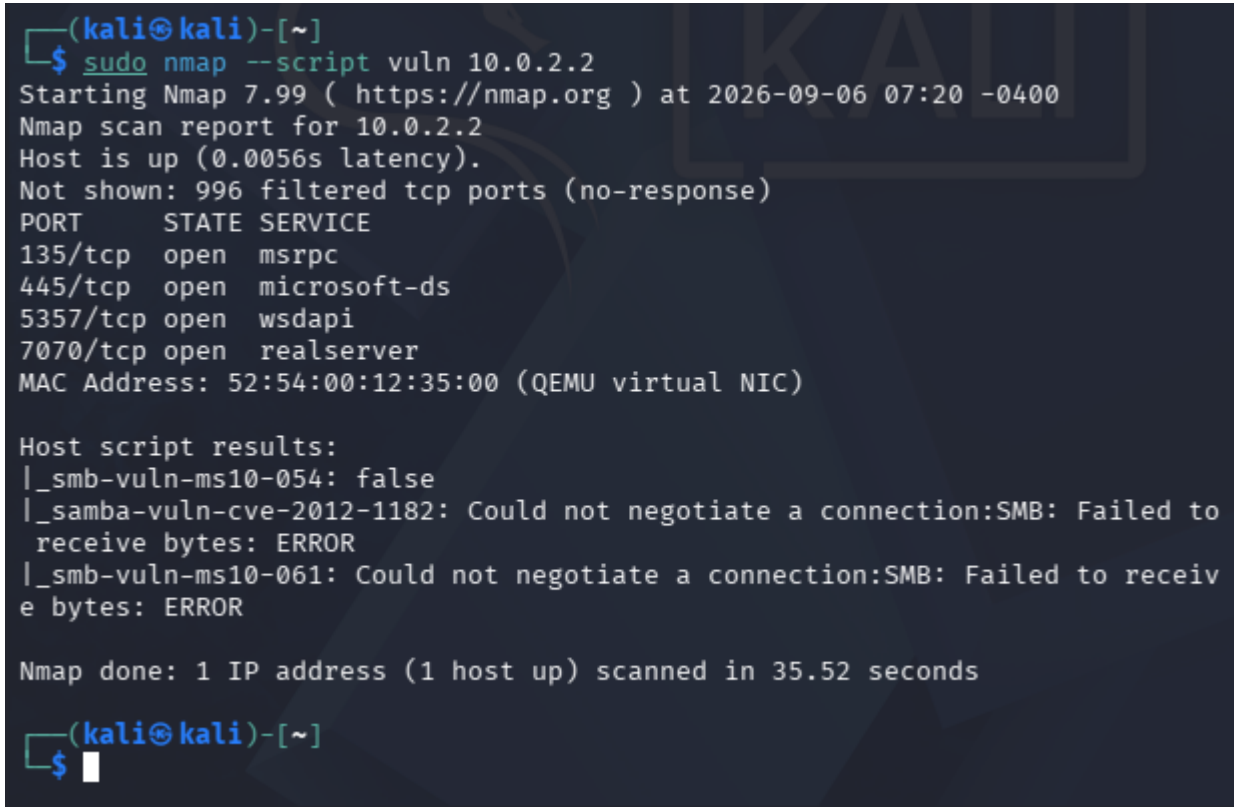
Nmap done: 1 IP address (1 host up) scanned in 9.93 seconds

(kali@kali)-[~]
$
```

Fig 9: nmap -sU --top-ports 20 UDP scan output

8.5 Vulnerability Scan (nmap --script vuln)

The NSE vulnerability script suite was run against the target. It attempted to check for SMB-related vulnerabilities (**MS10-054**, **CVE-2012-1182**, **MS10-061**). The smb-vuln-ms10-054 script returned 'false' (not vulnerable), while the other two scripts could not complete their checks and returned connection/negotiation errors. No confirmed vulnerabilities were identified in this scan; the errors are consistent with the target being a NAT gateway rather than a genuine file-sharing host.



```
(kali㉿kali)-[~]
└─$ sudo nmap --script vuln 10.0.2.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 07:20 -0400
Nmap scan report for 10.0.2.2
Host is up (0.0056s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp   open  msrpc
445/tcp   open  microsoft-ds
5357/tcp  open  wsdapi
7070/tcp  open  realserver
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)

Host script results:
|_smb-vuln-ms10-054: false
|_samba-vuln-cve-2012-1182: Could not negotiate a connection:SMB: Failed to
receive bytes: ERROR
|_smb-vuln-ms10-061: Could not negotiate a connection:SMB: Failed to receiv
e bytes: ERROR

Nmap done: 1 IP address (1 host up) scanned in 35.52 seconds

(kali㉿kali)-[~]
└─$
```

Fig 10: nmap --script vuln scan output

IP Address	Port	Protocol	Service	Version	Risk	Observation
10.0.2.2	135	TCP	msrpc	Microsoft Windows RPC	Medium	RPC endpoint mapper exposed; commonly targeted in Windows exploits
10.0.2.2	445	TCP	microsoft-ds	SMB (unspecified build)	High	SMB file sharing exposed; historically linked to major exploits (e.g. EternalBlue)
10.0.2.2	5357	TCP	wsdapi	Microsoft HTTPAPI httpd 2.0	Low	Used for Web Services Discovery (SSDP/UPnP); limited direct risk
10.0.2.2	7070	TCP	realserver	ssl/realserver (uncertain)	Medium	Uncommon port; unclear legitimate purpose in this context, warrants review

Advanced Analysis Per Port

Port 135 (msrpc): This is the Microsoft RPC endpoint mapper, used by Windows to coordinate remote procedure calls between services. It is required for various Windows networking features to function. It does not need to be exposed to untrusted networks; historically it has been a target for worms and remote exploits. If vulnerable, it could allow remote code execution. It should be restricted with a firewall to trusted hosts only, or disabled if not needed.

Port 445 (microsoft-ds / SMB): This port handles SMB file and printer sharing on Windows networks. It is required for file sharing between Windows machines on a local network. It should never be exposed to the internet or untrusted networks. If the underlying SMB version is outdated or unpatched, it could be exploited for remote code execution (as seen in real-world ransomware attacks). It should be firewalled from external access and kept fully patched.

Port 5357 (wsdapi): This port supports Web Services for Devices (WSD), used for device discovery (e.g., printers) via SSDP/UPnP. It is generally only needed on local/trusted networks. Exposure could allow information disclosure about the host or network. It should be disabled on servers that do not need device discovery features.

Port 7070 (realserver): This port is associated with streaming media services, though its presence here is unusual and warrants investigation to confirm which process is actually using it. Since its purpose could not be confirmed, it is recommended to verify its use and close it if it is not intentionally required.

9. Wireshark Methodology (Task 9)

Wireshark was launched on the Kali Linux machine and a live capture was started on interface eth0. To generate a representative mix of traffic, the following actions were performed from a terminal while the capture was running: an ICMP ping to an external host (**8.8.8.8**), a DNS lookup for google.com using nslookup, and an ICMP ping to the local gateway (**10.0.2.2**). This generated ARP, DNS, and ICMP traffic that was captured and analyzed.

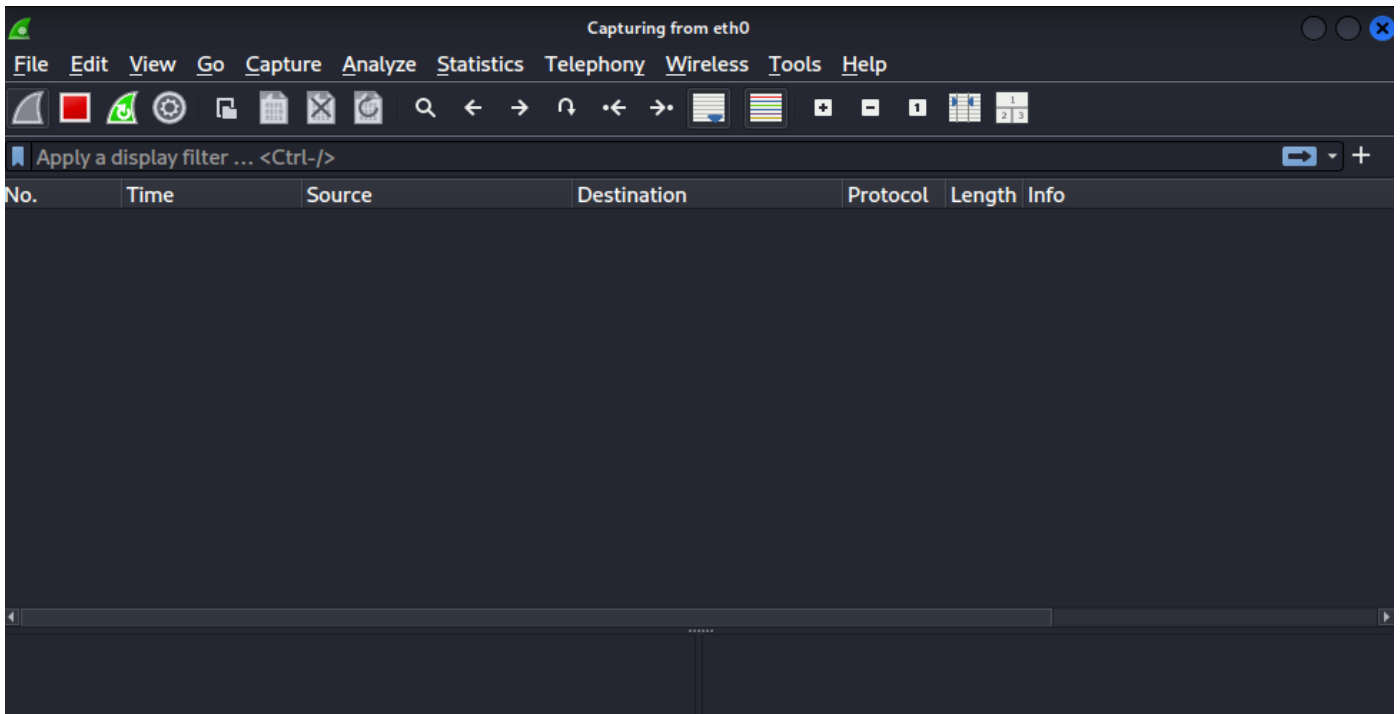


Fig 11: Wireshark capture started on eth0

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ping -c 5 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=64 time=21.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=22.6 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=21.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=21.8 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=64 time=20.7 ms

— 8.8.8.8 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4041ms
rtt min/avg/max/mdev = 20.747/21.742/22.620/0.595 ms
```

Fig 12: ping -c 5 8.8.8.8 output


```
(kali㉿kali)-[~]
$ nslookup google.com
Server: 10.0.2.3
Address: 10.0.2.3#53
Source: PCSSystemtec_5a:87:...
Destination: Broadcast
Non-authoritative answer:
Name: google.com
Address: 142.250.29.138
Name: google.com
Address: 142.250.29.113
Name: google.com
Address: 142.250.29.102
Name: google.com
Address: 142.250.29.100
Name: google.com
Address: 142.250.29.139
Name: google.com
Address: 142.250.29.101
Name: google.com
Address: 2404:6800:4013:804::64
Name: google.com
Address: 2404:6800:4013:804::65
Name: google.com
Address: 2404:6800:4013:804::8a
Name: google.com
Address: 2404:6800:4013:804::66
```

Fig 13: nslookup google.com output

```
(kali㉿kali)-[~]
$ ping -c 3 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data:
64 bytes from 10.0.2.2: icmp_seq=1 ttl=64 time=0.309 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=64 time=0.246 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=64 time=0.216 ms

— 10.0.2.2 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
rtt min/avg/max/mdev = 0.216/0.257/0.309/0.038 ms

(kali㉿kali)-[~]
$
```

Fig 14: ping -c 3 10.0.2.2 output confirming gateway connectivity

Filters used to analyze the capture included: dns, tcp, udp, icmp, arp, tcp.flags.syn==1, tcp.flags.reset==1, and dns.flags.response==0. The 'ping -c 5 8.8.8.8' command confirmed successful connectivity with **0% packet loss** (round-trip time averaging 21.7ms). The 'nslookup google.com' command resolved the domain to multiple IPv4 and IPv6 addresses via the lab's DNS resolver at **10.0.2.3**.

10. Packet Analysis (Task 9)

Timestamp	Source IP	Destination IP	Protocol	Src Port	Dst Port	Observation
241.387s	10.0.2.15 (bcast)	Broadcast	ARP	-	-	ARP request: 'Who has 10.0.2.3? Tell 10.0.2.15' - normal address resolution
241.388s	10.0.2.3	10.0.2.15	ARP	-	-	ARP reply confirming MAC address for 10.0.2.3
241.388s	10.0.2.15	10.0.2.3	DNS	random	53	Standard query: A record lookup for google.com
241.444s	10.0.2.3	10.0.2.15	DNS	53	random	Standard query response: A google.com resolved to public IP (142.x.x.x)
270.883s	10.0.2.15	10.0.2.2	ICMP	-	-	Echo (ping) request to gateway, seq=1
270.883s	10.0.2.2	10.0.2.15	ICMP	-	-	Echo (ping) reply from gateway, seq=1, round-trip essentially instant

The capture shows a typical, healthy sequence: an ARP request/reply pair to resolve a MAC address, followed by a DNS query/response pair to resolve a domain name, followed by ICMP echo request/reply pairs confirming reachability to both the local gateway and, separately, an external host (8.8.8.8). No unexpected resets, retransmissions, or unanswered queries were observed in this segment of the capture.

Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
16	241.387937995	PCSSystemtec_5a:87:...	Broadcast	ARP	42	Who has 10.0.2.3? Tell 10.0.2.15
17	241.388241387	52:54:00:12:35:00	PCSSystemtec_5a:87:...	ARP	64	10.0.2.3 is at 52:54:00:12:35:00
18	241.388247866	10.0.2.15	10.0.2.3	DNS	70	Standard query 0x9079 A google.com
19	241.444398907	10.0.2.3	10.0.2.15	DNS	166	Standard query response 0x9079 A google.com A 142.
20	241.445276919	10.0.2.15	10.0.2.3	DNS	70	Standard query 0x768c AAAA google.com
21	241.453403192	10.0.2.3	10.0.2.15	DNS	182	Standard query response 0x768c AAAA google.com AAA
22	270.883648564	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x83e6, seq=1/256, ttl=64
23	270.883944480	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x83e6, seq=1/256, ttl=64
24	271.901589785	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x83e6, seq=2/512, ttl=64
25	271.901813224	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x83e6, seq=2/512, ttl=64
26	272.918600209	10.0.2.15	10.0.2.2	ICMP	98	Echo (ping) request id=0x83e6, seq=3/768, ttl=64
27	272.918800839	10.0.2.2	10.0.2.15	ICMP	98	Echo (ping) reply id=0x83e6, seq=3/768, ttl=64
28	275.891869896	PCSSystemtec_5a:87:...	52:54:00:12:35:00	ARP	42	Who has 10.0.2.2? Tell 10.0.2.15
29	275.892248788	52:54:00:12:35:00	PCSSystemtec_5a:87:...	ARP	64	10.0.2.2 is at 52:54:00:12:35:00

Frame 1: Packet, 134 bytes on wire (1072 bits), 134 bytes captured on interface eth0

Ethernet II, Src: 52:54:00:12:35:00 (52:54:00:12:35:00), Dst: 01:00:5e:00:00:00

Internet Protocol Version 6, Src: fe80::2, Dst: ff02::1

Internet Control Message Protocol v6

0000 33 33 00 00 00 01 52 54 00 12 35 00 86 dd 60 00 33 ... R

0010 00 00 00 50 3a ff fe 80 00 00 00 00 00 00 00 00 ... P:

0020 00 00 00 00 00 02 ff 02 00 00 00 00 00 00 00 00 ...

0030 00 00 00 00 00 01 86 00 d7 03 40 00 07 08 00 00 ...

0040 00 00 00 00 00 00 01 01 52 54 00 12 35 00 03 04 ...

0050 40 c0 00 01 51 80 00 00 38 40 00 00 00 00 fd 17 @...Q

0060 62 5c f0 37 00 02 00 00 00 00 00 00 00 00 19 03 b\...7

0070 00 00 00 12 4f 80 fd 17 62 5c f0 37 00 02 00 00 ...0

0080 00 00 00 00 00 03

Fig 15: Packet list — ARP→DNS→ICMP sequence

11. Nmap + Wireshark Investigation (Task 10)

An Nmap SYN scan (**nmap -sS 10.0.2.2**) was executed against the authorized lab target while Wireshark was simultaneously capturing traffic on eth0, in order to observe how the scan appears at the packet level. The scan confirmed the same 4 open ports found earlier (135, 445, 5357, 7070), with 996 ports reported as filtered.

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo nmap -sS 10.0.2.2
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 08:47 -0400
Nmap scan report for 10.0.2.2
Host is up (0.0029s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
5357/tcp   open  wsddapi
7070/tcp   open  realserver
MAC Address: 52:54:00:12:35:00 (QEMU virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 7.29 seconds
  
```

Fig 16: nmap -sS scan running (Task 10)

No.	Time	Source	Destination	Protocol	Length	Info
4030	693.456897936	10.0.2.2	10.0.2.15	TCP	60	9220 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4031	693.456898101	10.0.2.2	10.0.2.15	TCP	60	2099 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4032	693.456996770	10.0.2.2	10.0.2.15	TCP	60	65129 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4033	693.457960543	10.0.2.2	10.0.2.15	TCP	60	5962 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4034	693.463346675	10.0.2.2	10.0.2.15	TCP	60	5904 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4035	693.463530018	10.0.2.2	10.0.2.15	TCP	60	1056 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4036	693.463530226	10.0.2.2	10.0.2.15	TCP	60	1900 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4037	693.463530385	10.0.2.2	10.0.2.15	TCP	60	4 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4038	693.463718178	10.0.2.2	10.0.2.15	TCP	60	57797 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4039	693.463718381	10.0.2.2	10.0.2.15	TCP	60	1277 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4040	693.463718514	10.0.2.2	10.0.2.15	TCP	60	11110 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4041	693.463810062	10.0.2.2	10.0.2.15	TCP	60	3580 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4042	693.463810214	10.0.2.2	10.0.2.15	TCP	60	4321 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0
4043	693.463934544	10.0.2.2	10.0.2.15	TCP	60	14442 → 47406 [RST, ACK] Seq=1 Ack=1 Win=65535 Len=0

Frame 1: Packet, 134 bytes on wire (1072 bits), 134 bytes captured on interface eth0

Ethernet II, Src: 52:54:00:12:35:00 (52:54:00:12:35:00), Dst: 02:00:00:00:00:00

Internet Protocol Version 6, Src: fe80::2, Dst: ff02::1

Internet Control Message Protocol v6

0000 33 33 00 00 00 01 52 54 00 12 35 00 86 dd 60 00 33 ...

0010 00 00 00 50 3a ff fe 80 00 00 00 00 00 00 00 00 ... P ...

0020 00 00 00 00 00 00 02 ff 02 00 00 00 00 00 00 00 ...

0030 00 00 00 00 00 00 01 86 00 d7 03 40 00 07 08 00 00 ...

0040 00 00 00 00 00 00 01 01 52 54 00 12 35 00 03 04 ...

0050 40 c0 00 01 51 80 00 00 38 40 00 00 00 00 fd 17 @ ... Q ...

0060 62 5c f0 37 00 02 00 00 00 00 00 00 00 00 19 03 b \ . 7 ...

0070 00 00 00 12 4f 80 fd 17 62 5c f0 37 00 02 00 00 ... 0 ...

0080 00 00 00 00 00 00 03

Fig 17: Wireshark capturing SYN scan (RST, ACK bursts)

During the scan, Wireshark captured a large burst of TCP packets between **10.0.2.2** and **10.0.2.15**. For each of the many closed/filtered ports probed, Nmap's SYN packet was answered by the target with a **RST, ACK** response, which is the standard TCP mechanism for signalling that a port is not accepting connections. A representative sample from the capture shows multiple RST, ACK packets arriving in quick succession from 10.0.2.2 to Kali's ephemeral port 47406, each corresponding to a different probed destination port (e.g. 9220, 2099, 65129, 5962, 5904, 1056, 1900, 57797, 1277, 11110, 3580, 4321, 14442), confirming that Nmap sequentially probed a large range of ports and received immediate RST responses for each closed one.

Nmap Result	Wireshark Evidence	Interpretation
Port detected open (135, 445, 5357, 7070)	SYN → SYN/ACK	Service appears reachable
Port detected closed (majority of scanned ports)	SYN → RST, ACK	Port is not accepting connections
Port not responding (filtered)	No expected response	Possible filtering/firewall

12. Vulnerability Assessment (Task 11 & 12)

12.1 Traffic Investigation Findings

Using Wireshark's Statistics > Conversations tool, the top communicating host pair was identified as 10.0.2.15 (Kali) and 10.0.2.2 (gateway/target), with **4,005 packets** and **237 KB** exchanged - dramatically higher than the other two conversations captured (10.0.2.15 to 8.8.8.8: 10 packets/980 bytes from the ping test; 10.0.2.15 to 10.0.2.3: 6 packets/727 bytes from the DNS query). This volume and pattern (thousands of packets to sequential destination ports in a short time window) is itself a signature of port scanning activity and would be a notable finding for a defender monitoring this network.

Wireshark · Conversations · eth0									
Conversation Settings									
☐ Name resolution ☐ Absolute start time ☐ Display raw data ☐ Limit to display filter Copy Follow Stream... Graph... I/O Graphs									
Protocol ☐ ILNP ☒ IPv4 ☒ IPv6 ☐ IPX ☐ JXTA Filter list for specific type									
Ethernet · 5	IPv4 · 3	IPv6 · 5	TCP · 1997	UDP · 4					
Address A	Address B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	
10.0.2.15	8.8.8.8	10	980 bytes	0	5	490 bytes	5	490 bytes	
10.0.2.15	10.0.2.2	4,005	237 kB	2	2,005	116 kB	2,000	120 kB	
10.0.2.15	10.0.2.3	6	727 bytes	1	3	221 bytes	3	506 bytes	

Fig 18: Wireshark Statistics → Conversations window

Time	Source IP	Destination IP	Port	Protocol	Observation	Security Significance
241.387s	10.0.2.15	Broadcast	-	ARP	ARP request for 10.0.2.3	Normal address resolution; not suspicious on its own
241.388s	10.0.2.15	10.0.2.3	53	DNS	Query for google.com	Normal outbound DNS; would warrant review only if domain were malicious
270.883s	10.0.2.15	10.0.2.2	-	ICMP	Ping to gateway	Routine connectivity check; low significance
693.456s+	10.0.2.2	10.0.2.15	many (9220, 2099, 65129, etc.)	TCP	Rapid burst of RST,ACK from one host across many sequential/random destination ports within milliseconds	Classic port-scanning signature; in a real environment this pattern from an external host would indicate reconnaissance and should trigger an alert
693.456s+	10.0.2.15	10.0.2.2	1000 ports	TCP	Corresponding high volume of SYN probes generating the conversation total of 4,005 packets/237KB	Confirms the scanning activity originated from the Kali host as part of this authorized assessment; in production this volume/pattern from an unrecognized host would be treated as an active attack indicator

Note: none of the traffic above is being labelled as malicious - all of it was generated intentionally as part of this authorized lab exercise. It is presented here to illustrate what scanning traffic looks like at the packet level and why the same pattern, if observed from an unauthorized source in a production network, would warrant investigation.

12.2 Security Findings

S.No.	Finding	Evidence	Risk	Potential Impact	Recommendation
1	SMB (port 445) exposed on the gateway/target host	Nmap -sV scan (Task 8)	High	SMB has historically been exploited for remote code execution and lateral movement (e.g. worm propagation)	Restrict SMB to trusted internal hosts only via firewall rules; disable if not required
2	RPC endpoint mapper (port 135) exposed	Nmap -sV scan (Task 8)	Medium	Can be leveraged to enumerate available RPC services and, if vulnerable, for remote exploitation	Firewall port 135 from untrusted networks; apply latest security patches
3	Unidentified service on port 7070 (realserver)	Nmap -sV / -sS scans (Task 7 & 8)	Medium	Unknown/unexpected services increase attack surface and are harder to monitor and patch	Investigate and confirm the purpose of this service; disable if unnecessary
4	UDP services responding as open filtered (67, 137, 4500)	Nmap -sU scan (Task 8)	Low	Ambiguous UDP responses could mask a genuinely open, unmonitored service	Manually verify each UDP service; disable NetBIOS (137) and DHCP server role (67) if not needed on this host
5	No confirmed patch-level vulnerabilities found, but SMB/RPC scripts could not fully negotiate a connection	Nmap --script vuln scan (Task 8)	Low	Incomplete vulnerability checks mean some issues may not have been detected due to environment limitations	Re-run vulnerability scans against a dedicated target host (not a NAT gateway) for more conclusive results

13. Risk Classification

Each finding from Section 12.2 has been classified based on the following reasoning:

Finding 1 - SMB (445) exposed - High: Classified High because SMB has a well-documented history of critical remote code execution vulnerabilities (e.g. EternalBlue/WannaCry). Even without a confirmed exploit in this scan, the mere exposure of this service carries significant inherent risk if left unpatched or unmonitored.

Finding 2 - RPC (135) exposed - Medium: Classified Medium because RPC is required for legitimate Windows networking functions and is less directly exploitable on its own, but historically has been used in combination with other services for lateral movement and worm propagation.

Finding 3 - Unidentified service on port 7070 - Medium: Classified Medium because the risk of an unknown service cannot be fully assessed - it may be benign or may represent unnecessary/misconfigured exposure. The uncertainty itself elevates the risk above Low until confirmed.

Finding 4 - Ambiguous UDP services (67, 137, 4500) - Low: Classified Low because these are common, often-expected UDP services in a NAT/DHCP environment, and 'open|filtered' results reflect scan ambiguity rather than confirmed exposure.

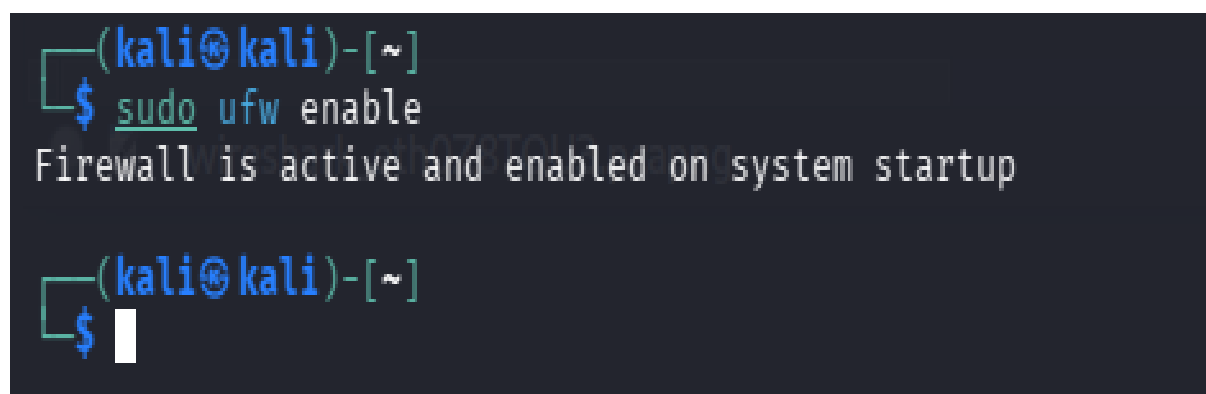
Finding 5 - Incomplete vulnerability scan results - Low: Classified Low because the errors encountered were due to environmental limitations (scanning a NAT gateway rather than a dedicated host) rather than an indication of an actual unpatched vulnerability.

14. Security Hardening (Task 13)

To demonstrate the hardening process, the SSH service on the Kali Linux analyst machine was used as a controlled example: it was intentionally started, confirmed as exposed via Nmap, then progressively hardened using a host-based firewall and service management, with each step re-verified via Nmap.

Improvement 1: Enable Host-Based Firewall (UFW)

UFW (Uncomplicated Firewall) was not installed by default on the Kali image, so it was installed with '**sudo apt install ufw -y**' and then activated with '**sudo ufw enable**'. This established a baseline firewall on the host, which is itself a hardening improvement since the machine previously had no host-based filtering active.

A terminal window with a dark background and light blue text. The prompt is '(kali@kali)-[~]'. The user enters '\$ sudo ufw enable'. The output is 'Firewall is active and enabled on system startup'. The prompt returns to '(kali@kali)-[~]'.

```
(kali@kali)-[~]  
$ sudo ufw enable  
Firewall is active and enabled on system startup  
(kali@kali)-[~]  
$
```

Fig 19: sudo apt install ufw + ufw enable

Improvement 2: Deny Rule for SSH Port

An explicit deny rule was added for port 22 using '`sudo ufw deny 22`', restricting SSH access at the firewall level.

```
(kali㉿kali)-[~]
$ sudo ufw deny 22
Rule added
Rule added (v6)
(kali㉿kali)-[~]
$
```

Fig 20: sudo ufw deny 22 rule added

Improvement 3: Disable the Unnecessary SSH Service

Since firewall rules do not reliably apply to loopback/local-host scans, the SSH service itself was stopped and disabled to permanently remove the exposure: '`sudo systemctl stop ssh`' followed by '`sudo systemctl disable ssh`' (which prevents the service from starting again on reboot). This is the most effective control, since it removes the unnecessary attack surface entirely rather than only filtering access to it.

```
(kali㉿kali)-[~]
$ sudo systemctl stop ssh

(kali㉿kali)-[~]
$ sudo systemctl disable ssh

(kali㉿kali)-[~]
$ sudo nmap 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 09:20 -0400
Stats: 0:00:52 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth S
can
SYN Stealth Scan Timing: About 24.00% done; ETC: 09:23 (0:02:35 remaining)
Stats: 0:00:53 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth S
can
SYN Stealth Scan Timing: About 24.50% done; ETC: 09:23 (0:02:34 remaining)
Nmap scan report for 10.0.2.15
Host is up.
All 1000 scanned ports on 10.0.2.15 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 205.79 seconds
(kali㉿kali)-[~]
$
```

Fig 21: systemctl stop/disable ssh + verification scan

15. Final Mini Security Assessment

This section consolidates all Week 3 activities into a single, real-world style security assessment, as if reporting to a team responsible for a small internal network.

Network Discovery

Three devices were identified on the lab network (10.0.2.0/24): the default gateway/NAT device (10.0.2.2), a secondary host (10.0.2.3), and the Kali Linux analyst machine (10.0.2.15). The gateway host exposed four TCP services: msrpc (135), microsoft-ds/SMB (445), wsdapi (5357), and an unidentified service (7070), along with several ambiguous UDP responses (67, 137, 4500).

Traffic Analysis

Captured traffic showed ARP address resolution, DNS query/response traffic for external domain lookups, and ICMP echo request/reply traffic to both the gateway and an external host. During an active Nmap SYN scan, a distinct high-volume burst of TCP SYN and RST,ACK packets was observed between the Kali machine and the gateway (4,005 packets / 237 KB in the busiest conversation) - a clear network-level signature of port scanning activity.

Security Assessment

Five findings were identified, with SMB exposure (**port 445**) rated as the highest risk due to its history of critical remote-code-execution vulnerabilities, followed by **RPC exposure and the unidentified port 7070 service rated Medium**. The ambiguous UDP findings and the inconclusive vulnerability-script results were rated Low. These ratings were supported directly by Nmap service/version detection output and NSE script results collected in Tasks 7-8.

Hardening

As a controlled demonstration, an exposed SSH service was hardened in three steps: installing and enabling a host-based firewall (UFW), adding an explicit deny rule for port 22, and finally disabling the SSH service itself. A follow-up Nmap scan confirmed that all 1000 scanned ports were no longer showing as open on the hardened host, demonstrating a measurable reduction in attack surface.

Final Recommendation

See Section 16 (Security Recommendations) for the five professional recommendations resulting from this assessment: restricting SMB/RPC exposure, investigating and disabling unidentified services, preferring service removal over firewall-only controls, enabling host-based firewalls by default, and re-scanning after every configuration change to verify hardening.

16. Before / After Comparison

Security Issue	Before	Action Taken	After	Improvement
No host-based firewall active on Kali	UFW not installed / inactive	Installed and enabled UFW ('apt install ufw', 'ufw enable')	Firewall active and enabled on system startup	Host now has a baseline firewall filtering unsolicited inbound traffic
SSH service unnecessarily exposed	nmap 10.0.2.15 showed port 22/tcp open (ssh)	Added firewall deny rule ('ufw deny 22')	Rule added successfully; port still showed open on local-host scan due to loopback scan limitations	Firewall-level control in place, though local scanning showed the need for a stronger control (see next row)
SSH service still running despite firewall rule	Port 22 open even after firewall deny rule (loopback scan limitation)	Stopped and disabled the SSH service directly ('systemctl stop ssh', 'systemctl disable ssh')	nmap 10.0.2.15 confirmed: 'All 1000 scanned ports on 10.0.2.15 are in ignored states' - port 22 no longer exposed	Attack surface fully reduced by removing the unnecessary service rather than only filtering it

17. Security Recommendations

Based on the findings of this assessment, the following professional recommendations are provided:

1. Restrict SMB (**port 445**) and RPC (**port 135**) access to trusted internal hosts only using firewall rules; these services should never be exposed to untrusted or external networks given their history of exploitation.
2. Disable any service that is not actively required for the host's function - for example, the unidentified service on port 7070 should be investigated and either justified or disabled.
3. Prefer disabling unused services entirely over relying solely on firewall rules, since firewall rules may not cover all traffic paths (e.g. loopback/local scans), as observed during the hardening exercise in this report.
4. Enable a host-based firewall (such as UFW) by default on all lab and production machines, with a default-deny posture for inbound connections and explicit allow rules only for required services.
5. Periodically re-run vulnerability and port scans after any configuration change to confirm that hardening actions have had the intended effect, as demonstrated in the before/after comparison in this report.

18. Challenges Faced

- Initial VirtualBox VM import issues: the Kali Linux .ova file did not import correctly on the first attempt, resulting in an empty virtual machine with no bootable disk ('No bootable medium found'). This was resolved by removing the failed VM and re-importing using File > Import Appliance rather than manually creating a new VM.
- Boot order misconfiguration: after resolving the import issue, a second VM had a valid disk attached but the boot order prioritized the optical drive over the hard disk. This was fixed via System > Motherboard > Boot Order in the VM settings.
- Downloaded an incompatible Ubuntu Server image (desktop-arm64) that could not run on the Intel/AMD-based host; the correct 'live-server-amd64' image was used instead.
- Firewall rules did not appear to affect Nmap scans run from the same machine (loopback scanning limitation) - this was worked around by stopping the underlying service directly rather than relying solely on firewall rules, which also served as a valuable learning point about the limits of host-based firewalls.
- An unexpected power outage interrupted the Wireshark capture session during Task 9, requiring the capture to be restarted; no data was lost from previously completed tasks since Nmap results had already been documented.

19. Conclusion

This Week 3 assessment provided hands-on experience with the full lifecycle of a basic network security assessment: discovering hosts, enumerating services and versions, capturing and interpreting live network traffic, correlating scanning activity with packet-level evidence, identifying security findings, classifying their risk, and applying and verifying hardening measures.

Before hardening, the lab environment exposed several services (**SSH, SMB, RPC, WSDAPI**, and an unidentified service on **port 7070**) that either should not have been exposed or required further investigation. After applying hardening measures - installing and enabling a host-based firewall and disabling an unnecessary service - the demonstrated SSH exposure was fully eliminated, as confirmed by a follow-up Nmap scan showing no open ports on the hardened host.

Overall, the assessment reinforced that automated scanning tools like Nmap and traffic analysis tools like Wireshark are complementary: Nmap efficiently identifies what is exposed, while Wireshark reveals how that exposure looks at the protocol level, which is valuable both for understanding attacker behavior and for verifying defensive controls.

20. References

- Nmap Official Website – <https://nmap.org>
- Nmap Reference Guide – <https://nmap.org/book/man.html>
- Nmap NSE Documentation – <https://nmap.org/book/nse.html>
- Wireshark Official Website – <https://www.wireshark.org>
- Wireshark User's Guide – https://www.wireshark.org/docs/wsug_html_chunked/
- Wireshark Display Filter Reference – <https://www.wireshark.org/docs/dfref/>
- MITRE ATT&CK – <https://attack.mitre.org>
- CVE Program – <https://www.cve.org>

21. Screenshots / Evidence

All screenshots and evidence referenced throughout this report (Fig 1–21) are embedded inline within their respective sections (Nmap Methodology, Nmap Results, Wireshark Methodology, Packet Analysis, Nmap + Wireshark Investigation, and Security Hardening). This section serves as a reference index — please see the corresponding figure numbers cited in Sections 6 through 14 for the complete visual evidence set.